

Spiegazione Homework #4 – MapReduce

I risultati degli script sono visibili sotto ogni blocco di celle

Considerazioni finali su implementazioni MapReduce e PySpark

Differenze

- **Modello di Programmazione:**

MapReduce (es. via MRJob) si basa su fasi rigide di map e reduce, con scrittura su disco tra le fasi. PySpark offre astrazioni più flessibili come RDD, DataFrames e SQL, permettendo pipeline in memoria e operazioni più complesse senza riutilizzo di codice (*boilerplate*) eccessivo.

- **Performance:**

PySpark (basato su Apache Spark) è generalmente più veloce (fino a 100 volte se lavora in-memory e fino a 10 se lavora con scritture su disco) grazie all'elaborazione in memoria e alla riduzione di I/O su disco. MapReduce è più lento per job iterativi o complessi.

- **Facilità d'Uso:**

PySpark è più intuitivo in Python, con API ad alto livello per trasformazioni e analisi. MapReduce richiede più codice manuale e gestione di cluster.

- **Ecosistema:**

PySpark integra con Spark (streaming, ML, graph processing), mentre MapReduce è limitato a elaborazione batch su Hadoop.

- **Scalabilità e Uso:**

Entrambi scalano su cluster, ma PySpark è adatto a big data interattivi e real-time; MapReduce è ideale per job semplici e affidabili su dati statici.

- **Uso personale:**

Personalmente ho preferito l'utilizzo di MapReduce per un semplice fattore di praticità ingegneristica, visto che è sufficiente seguire un workflow (map-combiner-reduce) molto semplice e di facile implementazione, anche traducendo lo pseudocodice trovato sulle slide.

Considerazioni quantitative

- **Tempi di esecuzione:**
 - **MR + Combiner:** 1.5754 secondi
 - **MR senza Combiner:** 1.4552 secondi
 - **Spark:** 1.1846 secondi

Spark è più veloce perché lavora in memoria ed evita continue scritture/letture su disco, mentre MapReduce fa passaggi rigidi (map → shuffle → reduce) che richiedono serializzazione e I/O. Il combiner in MapReduce tenta di ridurre la quantità di dati inviati al reduce, ma su dataset piccoli o con poche chiavi ripetute il sovraccarico di avvio, serializzazione e controllo può diventare maggiore del beneficio; quindi, diventa più lento anche se teoricamente “ottimizzato”.

Considerazioni qualitative

- PySpark è più “declarativo”, con trasformazioni a livello di DataFrame/RDD (map, filter, groupBy, agg) e chain fluide, quindi il codice è più compatto, leggibile e simile a Pandas/SQL. È facile mantenere e estendere per task di ETL/analytics.
- MapReduce (MRJob) è più “imperativo” e low-level: scrivi classi/mapper/reducer, gestisci input/output chiave-valore, spesso più boilerplate e più riga di codice per lo stesso risultato. È trasparente ma richiede gestione esplicita di serializzazione, parsing e logica di combinazione; più verboso e meno immediato da adattare a query complesse.

Considerazioni finali

Sebbene MapReduce sia caratterizzato da un paradigma a più basso livello rispetto a Spark, questo aspetto si allinea particolarmente bene con il mio approccio metodologico di sviluppo. L'architettura consente di mantenere una visibilità completa su ogni fase del processo di elaborazione: dal parsing iniziale del file CSV, alla selezione e trasformazione delle colonne, fino alla gestione esplicita del flusso di lavoro complessivo.

Spark, sebbene offra un'integrazione più naturale con costrutti SQL-Like (strumento che utilizzo frequentemente) nasconde le ottimizzazioni dietro astrazioni di alto livello e catene di funzioni le cui implementazioni interne non sempre sono immediatamente evidenti. Questa opacità comportamentale, pur migliorando le performance, può ostacolare la comprensione dettagliata del flusso dati durante la fase di sviluppo e manutenzione.